
Syllabus – Techniques logicielles pour le Cloud computing (TLC)

Responsable de cours : Brice Ekane

Contact : brice.ekane@univ-rennes.fr

Acronyme : TLC

Nombre de crédits : 3 ECTS

Volume horaire indicatif : 16.5h CM +16.5h TP

PRÉREQUIS

1. Connaissances en développement logiciel

Cycle de vie d'une application (développement, tests, déploiement). Notions de base en gestion de versions (ex : Git). Architecture logicielle (client/serveur, REST).

2. Connaissances de base en systèmes d'exploitation

Unix/Linux : ligne de commande, processus, services.

3. Réseaux et protocoles

TCP/IP, HTTP, DNS, pare-feux, ports.

DESCRIPTION DU COURS

Ce cours invite les étudiants à découvrir l'univers de l'Ops, souvent méconnu mais pourtant essentiel dans l'approche DevOps. À travers une progression thématique structurée, il met en lumière les enjeux liés à l'automatisation, au déploiement, à l'orchestration et à la supervision des applications modernes, dans un contexte cloud et cloud-native.

L'apprentissage commence par les fondements du cloud computing et évolue vers l'usage de technologies clés telles que Docker pour la conteneurisation, Ansible pour l'automatisation de l'infrastructure, Kubernetes pour l'orchestration de services et Quarkus pour le développement d'applications cloud-native performantes. L'approche pédagogique repose sur des cours illustrés, des vidéos ciblées, des travaux pratiques et un projet fil rouge, permettant aux étudiants de consolider les acquis par l'expérimentation.

Les notions de reverse proxy, de sécurité serveur, de gestion de configuration, ou encore d'observabilité (logs, métriques, traces) sont abordées dans un cadre concret, avec pour

objectif de fournir une compréhension fine des bonnes pratiques en matière de mise en production et de maintenance des applications. Le cours permet également de découvrir la méthodologie 12-factor et de mieux appréhender ce que signifie réellement le terme cloud-native dans une architecture applicative moderne.

Ce cours s'adresse à toute personne souhaitant développer une vision complète et opérationnelle de l'exploitation des applications métiers, à la croisée des chemins entre développement logiciel, administration système et ingénierie cloud.

PLACE DANS LE PROGRAMME D'ÉTUDE

Le cours **TLC – Techniques logicielles pour le Cloud computing** est une **unité d'enseignement transverse** du **semestre 1 du Master 2 Réseaux**, située à l'intersection du développement logiciel, de l'administration système et de l'ingénierie cloud.

Il complète les compétences réseau acquises dans les modules comme **ARES** (infrastructure, services réseau) et **ACI (ACI-Advanced cloud architectures)** en **apportant une approche orientée production logicielle** :

- Il prépare les étudiants à **automatiser le déploiement** d'applications via des outils comme **Ansible** ou **Terraform** ;
- Il initie aux pratiques d'**orchestration moderne** avec **Docker** et **Kubernetes**, en lien avec les enjeux de scalabilité, de haute disponibilité et de maintenabilité ;
- Il introduit les **bonnes pratiques DevOps** en matière de **configuration, supervision, sécurité**, et de développement **cloud-native** (méthodologie 12-factor, CI/CD, observabilité).

TLC offre ainsi aux étudiants les bases pour comprendre, concevoir et maintenir des architectures applicatives modernes dans des environnements **cloud publics, privés ou hybrides**, et les outille pour travailler efficacement en contexte **DevOps**

OBJECTIFS d'APPRENTISSAGE *(plus tard pour ceux qui sont prêts, rajouter les compétences)*

- **Comprendre** le mouvement DevOps, ses principes et enjeux dans le contexte du cloud.
- **Maîtriser** les outils modernes de déploiement, d'automatisation et de supervision pour applications cloud-native.
- **Expérimenter** les bonnes pratiques de développement, de test, de livraison et de maintenance d'applications sur le cloud.
- **Mettre en œuvre** une chaîne de déploiement automatisée d'une application cloud-native (projet fil rouge).

Compétences:

- Vision système complète d'une architecture cloud.
- Capacité à articuler outils, couches techniques et objectifs de production.
- Savoir préparer et faire vivre une plateforme cloud-native.

EXIGENCE DE TRAVAIL (du mois de fin janvier au mois de mis-février)

En présentiel	Travail personnel
36h	

PROGRAMME

Cours magistraux

Séance	Thème	Objectifs/Contenu
1	Introduction au Cloud Computing	Définition, historique, enjeux, architectures, notion de cloud-native
2	Modern Development s & DevOps	Principes du DevOps, état de l'art, automatisation, monitoring
3	Docker & les conteneurs	Conteneurisation, microservices, workflow Docker
4	Microservices & API Management	Conception microservices, gestion et sécurité des API, continuous delivery
5	Infrastructure as Code (IaC)	Automatisation de l'infra, Ansible, pratiques et outils
6	Méthodologie 12 Factors	Bonnes pratiques du cloud-native, portabilité, scalabilité, déploiement continu
7	Kubernetes	Orchestration de conteneurs, architecture K8s, déploiement, tutoriels
8	Web App Ops	Déploiement d'applications web, reverse proxy, DNS, sécurité serveur, automatisation
9	Quarkus et apps Java cloud-native	Quarkus, GraalVM, exécution native, cas d'usage
10	Monitoring et	Prometheus, Grafana, ELK stack, Datadog, supervision

Travaux Pratiques

- **TP1 : Docker**
 - Prise en main, création d'images, déploiement, Load Balancer, docker-compose
- **TP2 : Ansible**
 - Déploiement automatisé, configuration, gestion d'environnements

Projet fil rouge

Déploiement automatisé d'une application "Doodle-like" intégrant un éditeur Etherpad, dans une chaîne cloud-native :

- Conteneurisation, orchestration, sécurisation (reverse proxy, firewall, SSL)
- Extension : CI/CD, monitoring avancé, orchestration avec Kubernetes, identity provider/API gateway

BIBLIOGRAPHIE

BALALAE, Armin, et al. "Microservices architecture enables devops: Migration to a cloud-native architecture." IEEE Software, 2016.

Linthicum, D. S., "Cloud-Native Applications and Cloud Migration: The Good, the Bad, and the Points Between," IEEE Cloud Computing, 2017.

MODALITÉS D'ÉVALUATION

- **TP Docker** (note individuelle)
- **TP Ansible** (note individuelle)
- **Projet** (note de groupe, rendu final avec rapport technique et soutenance)
- **Participation et questionnements** (bonus)
- **Contrôle continu**